A Survey of Higher-Order Structures in Graph Learning and Associated Message Passing Frameworks

Foundational notes for higher-order message passing and random-walk-based propagation

Adrián Ruiz Doblas

Abstract

Graph Neural Networks (GNNs) provide a principled way to learn from relational data supported on graphs, respecting permutation symmetries and enabling local-to-global representation learning via message passing. However, many real-world systems exhibit *polyadic* (multiway) interactions and *multi-rank* relational organization that cannot be faithfully reduced to pairwise edges. This survey gives a self-contained overview of higher-order domains used in graph learning (hypergraphs, simplicial complexes, cellular complexes, and combinatorial complexes) and of the message passing frameworks developed for them. We begin from the most basic graph-theoretic definitions and the MPNN formalism on graphs, then build higher-order generalizations in a unified operator-centric language: message passing is a choice of carrier objects, neighborhood relations, and propagation/transport operators. This perspective makes random walks and diffusion kernels appear naturally as propagation operators and highlights the design degrees of freedom that matter for follow-up research. We conclude by synthesizing an open question motivating subsequent work: for hypergraphs, can one define a genuinely hypergraph-native message passing paradigm based directly on hyperedge connectivity patterns and hypergraph-specific random-walk geometries in the spirit of Eidi and Otter [3] rather than via relational or auxiliary graph constructions as in Taha et al. [11]?

Contents

1 Introduction and scope

1.1 Why higher order?

Graph learning studies data where entities are coupled by relations. Graphs encode dyadic relations; in many domains, the natural relations are *polyadic* (groups) or *multi-rank* (nodes-edges-faces-cells). As a canonical example, "three authors writing one paper together" and "three separate pairwise collaborations" both project to the same triangle graph, but are different polyadic patterns. This motivates *higher-order graph data models* (HOGDMs) and *higher-order graph neural networks* (HOGNNs) [1, 5].

(a) One 3-way collaboration

(b) Three separate pairwise collaborations

Figure 1: Two different collaboration patterns on the same three authors. Left: a single polyadic interaction (one joint paper). Right: three distinct dyadic interactions. Both may project to the same triangle graph, but they encode different higher-order structure.

1.2 What this document covers

This survey is the foundational deliverable for a broader project plan that later focuses on random-walk variants and their lifts to higher-order domains. Accordingly, we focus on:

- **Domains:** hypergraphs, simplicial complexes, cellular complexes, combinatorial complexes, and related relational representations.
- **Message passing frameworks:** node- and rank-based message passing, incidence-based propagation, Hasse-graph-based constructions, and (co)sheaf-like transport.
- **Propagation operators:** how random walks and diffusions instantiate the linear mixing component in message passing, and what changes in higher-order domains.

We do *not* cover implementation or experiments here; we only prepare the mathematical and conceptual ground.

1.3 Primary references and guiding question

We build around the surveys/framework papers Besta et al. [1], Hajij et al. [5, 6], Papillon et al. [10] and the axiomatic connection between simplicial message passing and *relational structures* Taha et al. [11]. For random walks on hypergraphs we rely on Carletti et al. [2], and for role-based motivation and hypergraph-sensitive neighborhood constructions we use Eidi and Otter [3]. Background on standard GNNs follows Gilmer et al. [4], Wu et al. [13].

Guiding open question. Two complementary philosophies are prominent:

- **Intrinsic higher-order propagation:** define neighborhoods and operators directly from the higher-order incidence/connectivity.
- **Relationalization:** represent the domain as a relational structure (multiple relations) or auxiliary graphs, and define message passing there.

For hypergraphs in particular, the question is whether intrinsic propagation operators derived from hyperedge connectivity patterns (including multi-step/restart variants) better capture *structural/regular roles* than one-step baselines, and how this compares to relational/auxiliary representations [3, 11].

2 Graphs and standard message passing

2.1 Graphs and attributed graphs

Definition 2.1 (Graph)**.** *A (finite, simple) undirected graph is a pair $G = (V, E)$ where V is a finite set of vertices and $E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}$ is a set of edges. A directed graph has $E \subseteq V \times V$.*

In graph learning, nodes and/or edges carry attributes. An attributed graph can be noted as $G = (V, E, X)$, where $X \in \mathbb{R}^{n \times F}$ is the node-feature matrix ($n = |V|$), and F denotes the number of node features, i.e., the dimensionality of each node-attribute vector $x_v \in \mathbb{R}^F$, whose v-th row x_v encodes the features of node v [13]. When node attributes are unavailable, it is common to set X to a constant (e.g. all-ones) to keep a uniform notation. A common alternative representation is (A, X) where $A \in \mathbb{R}^{n \times n}$ is the adjacency matrix (in the unweighted case, $A_{ij} \in \{0, 1\}$ indicates whether an edge is present; in weighted graphs, $A_{ij} \in \mathbb{R}_+$ stores an edge weight, and absent edges correspond to $A_{ij} = 0$). For undirected graphs, the adjacency matrix is symmetric ($A_{ij} = A_{ji}$). For directed graphs, A_{ij} represents an edge with source v_i and target v_j, so A need not be symmetric. Edge features can be stored as tensors $E \in \mathbb{R}^{n \times n \times K}$, or via typed relations $\{A^{(r)}\}_{r \in \mathcal{R}}$ in multi-relational graphs.

Example 2.1 (Molecular graphs)**.** *A canonical instance of attributed graphs is molecular data: nodes are atoms with node-type features, while bonds are edges with bond-type features. One convenient encoding is an adjacency tensor $A \in \mathbb{R}^{n \times n \times Y}$ whose Y slices correspond to bond types, together with a node-type matrix $X \in \mathbb{R}^{n \times T}$ whose T columns correspond to atom types.*

Definition 2.2 (Neighborhoods and degrees)**.** *For a node $v \in V$, its (1-hop) neighborhood is*

$$N(v) := \{u \in V : \{u, v\} \in E\} \quad \textit{(or } N(v) := \{u \in V : (u, v) \in E\} \textit{ in the directed case),}$$

and its degree is $\deg(v) := |N(v)|$*. The k-hop neighborhood $N^k(v)$ is the set of nodes at graph distance at most k from v.*

2.2 Linear algebra: degrees and Laplacians

We now recall the basic linear-algebraic operators that encode graph connectivity and that will later reappear as propagation operators in message passing.

Definition 2.3 (Degree matrix)**.** *Let A be the (possibly weighted) adjacency matrix of a graph on n nodes. The* degree matrix *is $D = \operatorname{diag}(d_1, \ldots, d_n)$, where*

$$d_i := \sum_j A_{ij}.$$

From the adjacency matrix A and the degree matrix D, one constructs the graph Laplacian, one of the central operators in graph theory. Intuitively, it compares the value of a node-wise signal at a node with the values at its neighbors, thereby measuring variation or lack of smoothness over the graph. This is why Laplacian-based operators naturally appear in diffusion processes, spectral methods, and later in message-passing architectures.

Example 2.2 (Common graph Laplacians)**.** *The most common Laplacian operators are:*

- *The* combinatorial (unnormalized) Laplacian $L = D - A$*: it is a central operator in spectral graph theory and can be interpreted as encoding a notion of smoothness of node-wise signals over the graph.*
- *The* random-walk Laplacian $L_{\mathrm{rw}} = I - D^{-1}A$*.*
- *The* symmetric normalized Laplacian $L_{\mathrm{sym}} = I - D^{-1/2}AD^{-1/2}$ *(see Appendix A for its derivation).*

These operators generate diffusions and appear in spectral perspectives on graph learning.

Remark 2.1 (Importance of the normalized Laplacian in GNNs)**.** *We can gain some intuition on why the normalized Laplacian is defined.*

- *For undirected graphs, the normalized Laplacian induces the quadratic form*

$$x^\top L_{\mathrm{norm}} x = \frac{1}{2} \sum_{i,j} A_{ij} \left(\frac{x_i}{\sqrt{d_i}} - \frac{x_j}{\sqrt{d_j}} \right)^2,$$

  which measures how different neighboring values are after degree correction.

- Spectral viewpoint: *since L_{norm} is symmetric, it admits an orthonormal eigenbasis (Spectral Theorem); its eigendecomposition provides a graph Fourier basis used to define spectral graph convolutions.*

- Message passing viewpoint: *the same normalization pattern appears in practice (e.g. GCN layers) to stabilize neighborhood aggregation and make feature magnitudes less dependent on node degree.*

2.3 Permutation symmetry

Unlike grid-structured data, graphs do not come with a canonical ordering of nodes: any relabeling of indices represents the same underlying object. Consequently, graph learning models should be compatible with node relabelings: graph-/set-level predictions should be invariant, while node-level predictions (e.g. embeddings) should be equivariant. This requirement is what ultimately forces aggregation operators (this is, a permutation-invariant function that combines the features or messages coming from a node's neighbors into a single summary representation) to ignore the ordering of neighbors to ignore the ordering of neighbors [9].

Definition 2.4 (Permutation action on node-indexed objects)**.** *Let π be a permutation of $\{1, \dots, n\}$ (a relabeling of node indices). It acts on node-indexed objects by reindexing:*

$$\begin{aligned} X &\mapsto \pi(X) \quad \textit{(permute rows)}, \\ A &\mapsto \pi(A) \quad \textit{(permute rows and columns)}, \\ S &\mapsto \pi(S), \end{aligned}$$

where $A \in \mathbb{R}^{n \times n}$ is the adjacency matrix, $X \in \mathbb{R}^{n \times d}$ the node-feature matrix, and $S \subseteq V$ a target node set.

Definition 2.5 (Permutation invariance (graph-/set-level))**.** *A function producing a graph-/set-level output is permutation invariant if, and only if,*

$$f(S, A, X) = f(\pi(S), \pi(A), \pi(X)) \qquad \forall \pi.$$

Definition 2.6 (Permutation equivariance (node-level))**.** *A function producing node-level outputs (e.g. embeddings) is permutation equivariant if, and only if,*

$$\pi\big(f(A, X)\big) = f(\pi(A), \pi(X)) \qquad \forall \pi,$$

i.e., outputs permute consistently with inputs.

Neighborhood information is inherently unordered (a multiset, see Definition 2.9), so any aggregation of neighbor messages must be permutation-invariant (e.g. sum/mean/max/attention pooling). This will be made explicit in the MPNN definition in the next section.

Definition 2.7 (Neural networks (minimal background))**.** *A (feed-forward) neural network is a parameterized function obtained by composing layers. A standard affine-nonlinear layer maps $z \in \mathbb{R}^d$ to*

$$z \mapsto \sigma(Wz + b),$$

where $W \in \mathbb{R}^{d' \times d}$ and $b \in \mathbb{R}^{d'}$ are learnable parameters and σ is an element-wise nonlinearity. Parameters are learned by minimizing a loss function, i.e. a scalar objective measuring prediction error on the training data, typically by variants of stochastic gradient descent, which update the parameters iteratively using gradient information computed on small batches.

2.4 Message Passing Neural Networks (MPNNs)

Message Passing Neural Networks (MPNNs) provide a common language for many GNN architectures. Their key design principle is to update node representations by exchanging information along edges, while respecting permutation symmetry (Section 2.3). We first recall the standard layer definition, then reformulate it in an operator-centric language that will later extend naturally to higher-order structures.

Definition 2.8 (MPNN layer 7)**.** *Let $h_v^{(\ell)} \in \mathbb{R}^d$ denote the latent representation of node v at layer ℓ, and let e_{uv} denote (optional) edge features. An MPNN layer computes a message from neighbors and updates each node via*

$$m_v^{(\ell+1)} = \mathrm{AGG}\Big(\{\mathrm{MLP}_e(h_v^{(\ell)}, h_u^{(\ell)}, e_{uv}) : u \in N(v)\}\Big), \tag{1}$$

$$h_v^{(\ell+1)} = U_\ell\big(h_v^{(\ell)}, m_v^{(\ell+1)}\big), \tag{2}$$

where $N(v)$ is the neighborhood of v, AGG *is a permutation-invariant aggregation map over multisets (e.g. sum, mean, max, attention pooling), and U_ℓ is a learnable update map.*

In simpler words, an MPNN layer has three ingredients:

- A *message function* that computes information sent from neighbors.
- A *permutation-invariant aggregation* of incoming messages.
- An *update map* that combines the previous node state with the aggregated neighborhood information.

This local viewpoint will later generalize to higher-order domains by changing the carrier objects and the propagation operator.

Remark 2.2 (Operator-centric view)**.** *Many popular architectures can be expressed in matrix form as*

$$H^{(\ell+1)} = \sigma\big(P_{\mathrm{prop}}\, H^{(\ell)} W^{(\ell)}\big),$$

where $H^{(\ell)} \in \mathbb{R}^{n\times d}$ stacks node representations, P_{prop} is a (normalized) propagation operator, and $W^{(\ell)}$ is learnable. This global form emphasizes that a layer consists of two conceptually distinct parts: a learnable feature transformation ($W^{(\ell)}$) and a propagation rule (P_{prop}). Choosing P_{prop} is central when moving from graphs to higher-order structures such as hypergraphs and complexes.

Remark 2.3 (Tasks, node embeddings, and readout)**.** *A GNN can be viewed as a map*

$$f : \mathbb{R}^{n\times n} \times \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d'}, \qquad (A, X) \mapsto Z,$$

where the i-th row of Z is the embedding of node v_i. Many learning tasks can be described by specifying a target set $S \subseteq V$ and predicting a label or value y_S associated with S. Special cases include node-level tasks ($|S| = 1$), edge-level tasks ($|S| = 2$), and graph-level tasks ($S = V$). For graph-level prediction, one applies a permutation-invariant readout

$$h_G = \mathrm{READOUT}\big(\{h_v^{(L)} : v \in V\}\big).$$

Definition 2.9 (Multisets and invariant pooling)**.** *A multiset is an unordered collection in which repetitions of elements are allowed. In message passing, neighborhood aggregation acts on multisets of incoming messages, so the aggregation rule must be invariant under reordering. Common choices are*

$$\sum_{u\in N(v)} m_{u\to v}^{(\ell)}, \qquad \frac{1}{|N(v)|}\sum_{u\in N(v)} m_{u\to v}^{(\ell)}, \qquad \max_{u\in N(v)} m_{u\to v}^{(\ell)}$$

(componentwise in the last case).

A standard template for permutation-invariant set functions is given by the Deep Sets representation

$$q(S) = \phi\left(\sum_{a \in S} \psi(a)\right),$$

where ψ encodes individual elements, the summation aggregates them in a permutation-invariant way, and ϕ maps the aggregated summary to the final output. This explains why sum-based pooling, followed by a learnable map, is a natural design pattern in MPNNs.

Remark 2.4 (Equivalent MSG/AGG/UPT notation)**.** *Equations (1)–(2) are often written by separating message computation, aggregation, and update. Initialize* $h_v^{(0)} := x_v$*. For* $\ell = 1, \dots, L$*:*

$$m_{u \to v}^{(\ell)} = \mathrm{MSG}^{(\ell)}(h_v^{(\ell-1)}, h_u^{(\ell-1)}, e_{uv}), \qquad u \in N(v); \tag{3}$$

$$a_v^{(\ell)} = \mathrm{AGG}^{(\ell)}\big(\{m_{u \to v}^{(\ell)} : u \in N(v)\}\big); \tag{4}$$

$$h_v^{(\ell)} = \mathrm{UPT}^{(\ell)}(h_v^{(\ell-1)}, a_v^{(\ell)}). \tag{5}$$

Typically, $\mathrm{MSG}^{(\ell)}$ *and* $\mathrm{UPT}^{(\ell)}$ *are parameterized (e.g. by MLPs), whereas* $\mathrm{AGG}^{(\ell)}$ *must be permutation-invariant.*

Remark 2.5 (Implementation notes.)**.** *A multilayer perceptron (MLP) is a standard feed-forward neural network, i.e. a composition of affine maps and pointwise nonlinearities; for example, a 2-layer MLP has the form*

$$\mathrm{MLP}(x) = W_2\, \sigma(W_1 x + b_1) + b_2.$$

In GNNs, MLPs are commonly used to parameterize the message and/or update maps.

Optionally, the update step may use a gated recurrent mechanism such as a GRU, viewing $h_v^{(\ell-1)}$ as a previous state and $a_v^{(\ell)}$ as an input. This allows the model to control how much of the previous state is retained versus overwritten.

A typical message passing layer scales approximately as $\mathcal{O}(|E|d)$, where d is the embedding dimension, since each edge contributes to message computation at most once per layer. It also encodes a specific inductive bias: after L layers, $h_v^{(L)}$ can depend only on nodes in the L-hop neighborhood of v. This locality is both a strength (scalability on sparse graphs) and a limitation.

2.5 Canonical Architectures as Instances of Message Passing

Before moving to higher-order domains, it is useful to see how several canonical GNN architectures fit into the MPNN template of Section 2.4. The point is not to give a full survey of graph architectures, but to highlight how different design choices (normalized linear propagation, sampled aggregation, or attention-weighted aggregation) arise as different instances of the same message passing paradigm. This perspective will be important later, when the main design question becomes how to choose the propagation operator in higher-order structures.

- **GCN**: Graph Convolutional Networks employ normalized linear aggregation, typically using $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ with self-loops, to stabilize training and account for varying degrees. A GCN layer [8] (with self-loops) is

  $$H^{(\ell)} = \sigma\Big(\tilde{D}^{-1/2}\tilde{A}\,\tilde{D}^{-1/2}H^{(\ell-1)}W^{(\ell)}\Big), \qquad \tilde{A} = A + I,$$

  which corresponds to a linear message, normalized sum aggregation, and pointwise nonlinearity.

- **GraphSAGE**. GraphSAGE [7] introduces neighborhood sampling and flexible aggregation functions (mean, pooling, LSTM), supporting inductive learning on large graphs, and it uses alternative aggregators and often sampling for scalability.

- **GAT**. Graph Attention Networks replace uniform aggregation weights by learned attention coefficients, allowing the model to focus on more relevant neighbors. GAT [12] learns attention coefficients $\alpha_{vu}^{(\ell)}$ on edges and uses a weighted aggregation:

$$a_v^{(\ell)} = \sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu}^{(\ell)} \, W^{(\ell)} h_u^{(\ell-1)}, \qquad h_v^{(\ell)} = \sigma(a_v^{(\ell)}),$$

  where the attention weights are obtained via a masked softmax over neighbors. This is again a message passing layer with a data-dependent aggregation operator.

2.6 Expressivity through the Weisfeiler-Leman lens

A standard way to analyse the distinguishing power of graph representation methods is through the Weisfeiler-Leman (WL) hierarchy. In particular, many MPNNs are naturally compared against the 1-dimensional Weisfeiler-Leman test (1-WL), also known as *color refinement*. This gives a precise benchmark for what standard neighborhood-aggregation architectures can and cannot distinguish [14].

Definition 2.10 (1-WL / color refinement)**.** *Let $G = (V, E)$ be a graph with an initial node labeling (or initial node features) $\{c_v^{(0)}\}_{v \in V}$. The 1-dimensional Weisfeiler–Leman procedure iteratively refines node colors according to*

$$c_v^{(t+1)} = \mathrm{HASH}\Big(c_v^{(t)}, \{\!\{\, c_u^{(t)} : u \in N(v) \,\}\!\}\Big),$$

where $\{\!\{\cdot\}\!\}$ denotes a multiset and HASH *is an injective encoding. Thus, two nodes receive the same new color if and only if they had the same previous color and the same multiset of neighbor colors.*

Remark 2.6. *To compare two graphs G and H, one runs the above refinement on both graphs and compares the multisets of colors produced at each iteration. If at some iteration these multisets differ, then G and H are certainly non-isomorphic. If the procedure stabilizes and the color histograms remain identical, then the graphs are 1-WL-indistinguishable. Hence 1-WL is an efficient graph isomorphism heuristic, but not a complete graph isomorphism algorithm.*

Remark 2.7. *The relevance of 1-WL in graph learning is that standard message passing architectures also update each node by combining its current state with a permutation-invariant aggregation of neighbor states. As a result, for standard permutation-invariant neighborhood-aggregation MPNNs, their ability to distinguish non-isomorphic graphs is upper-bounded by 1-WL. Conversely, with sufficiently expressive injective aggregation and update maps, one can match the power of 1-WL [14].*

Example 2.3. *A canonical neural architecture achieving this bound is the Graph Isomorphism Network (GIN), whose layer has the form*

$$h_v^{(\ell+1)} = \mathrm{MLP}^{(\ell)}\left((1 + \varepsilon^{(\ell)})\, h_v^{(\ell)} + \sum_{u \in N(v)} h_u^{(\ell)}\right).$$

With sufficiently expressive multilayer perceptrons and injective multiset aggregation, GIN matches the discriminative power of 1*-WL. This makes* 1*-WL the natural expressivity baseline for ordinary MPNNs [14].*

Beyond color refinement, the first nontrivial strengthening is the 2-dimensional Weisfeiler–Leman test (2-WL), and more generally the k-dimensional WL hierarchy.

Definition 2.11 (k-WL)**.** *For an integer $k \geq 2$, the k-dimensional Weisfeiler-Leman test assigns colors to ordered k-tuples of vertices rather than to individual vertices. At each iteration, the color of a k-tuple is updated by combining its current color with the multiset of colors of neighboring k-tuples obtained by replacing one coordinate at a time. The case $k = 2$ already yields a strictly stronger test than* 1*-WL, and more generally increasing k produces a hierarchy of increasingly expressive graph distinguishing procedures.*

Remark 2.8 (WL-inspired higher-order architectures)**.** *The WL hierarchy has motivated several higher-order architectures, as reviewed by Besta et al. [1]. In particular, k-GNNs lift hidden states from individual nodes to k-node subsets and perform message passing between overlapping subsets, thereby providing a neural analogue of higher-order WL refinement. Likewise, invariant and equivariant higher-order graph networks operate on higher-order tensors and are motivated by the same goal of overcoming the expressivity limitations of standard* 1*-WL-level message passing while preserving permutation symmetry [9].*

One natural way to move beyond the 1-WL barrier is to enlarge the carrier of hidden states from individual nodes to tuples or sets of nodes. This leads to a second notion of "higher order" in graph learning: instead of changing the underlying domain itself (for instance, from graphs to hypergraphs or complexes), one keeps the graph as the base object but lifts the hidden representations from nodes to tuples or subsets of nodes. This viewpoint provides a natural bridge to tuple-based higher-order message passing architectures.

2.7 Tuple-based higher-order message passing

Here the underlying graph remains the same, but hidden states are attached to higher-order entities such as k-tuples or k-sets of nodes. Message passing is then defined between these derived objects rather than only between original graph nodes.

Definition 2.12 (Higher-order message passing (tuples/sets))**.** *Simple message passing stores hidden states on nodes (1-tuples) and communicates along edges. Higher-order message passing generalizes the carrier of representations from nodes to k-tuples or k-sets of nodes and performs message passing between such higher-order entities, mirroring higher-dimensional WL tests.*

Remark 2.9 (The k-WL hierarchy (intuition))**.** *The k-WL family maintains labels on k-tuples and updates them through neighborhoods defined by replacing coordinates. Increasing k yields a strict hierarchy of increased distinguishing power (for $k \geq 2$). Intuitively, moving from nodes to tuples allows the model to represent structured configurations that are invisible to purely node-based message passing.*

Definition 2.13 (k-GNN (WL-induced higher-order GNN))**.** *A prototypical higher-order message passing architecture assigns an embedding $h_j^{(\ell)}$ to each k-node set $V_j \subseteq V$ and updates by aggregating*

over k-sets that overlap in $k-1$ nodes

$$h_j^{(\ell+1)} = \mathrm{MLP}\left(h_j^{(\ell)} \oplus \sum_{V_j' \in N_{k\text{-GNN}}(V_j)} h_{j'}^{(\ell)}\right), \qquad N_{k\text{-GNN}}(V_j) := \{V_j' : |V_j' \cap V_j| = k-1\}. \tag{6}$$

Equivalently, this is message passing on a derived graph whose nodes are k-sets and whose edges connect sets with overlap $k-1$.

Remark 2.10 (Computational trade-off)**.** *Tracking all k-tuples/sets is expensive: in general, higher-order methods require $\Omega(n^k)$ memory and $\Omega(n^{k+1})$ computation, which quickly becomes prohibitive as k grows. This motivates approximations such as sampling, locality restrictions, sparsification, or domain-specific choices of higher-order carriers.*

Remark 2.11 (Tensor-based higher-order networks)**.** *Another approach represents k-tuple features as order-k tensors and constructs permutation invariant/equivariant maps directly at the tensor level. This viewpoint underlies provably powerful higher-order architectures such as PPGN.*

2.8 Limitations: oversmoothing and oversquashing

While simple message passing is effective, it has well-known limitations that motivate richer constructions. Two particularly common failure modes are *oversmoothing* and *oversquashing* [11].

Definition 2.14 (Oversmoothing)**.** *Oversmoothing refers to the phenomenon that repeated neighborhood averaging can drive node embeddings to become nearly constant within connected components, degrading discriminability as depth increases.*

Definition 2.15 (Oversquashing)**.** *Oversquashing refers to the phenomenon that exponentially many distant messages (or long-range dependencies) may be compressed through narrow neighborhood bottlenecks into fixed-size representations, limiting the ability to transmit information across the graph.*

Higher-order domains and/or higher-order propagation can mitigate these issues by changing connectivity and the geometry of information flow [5, 11].

Some structures (e.g. cycles, motifs, simplicial interactions, or hyperedges) are not naturally captured when the model state is associated only with individual nodes and messages pass only along edges. This motivates higher-order carriers, higher-order domains, and/or richer propagation operators.

3 Higher-order graph data models (HOGDMs)

3.1 Taxonomy: set-type vs hierarchical relations

A useful organizing axis is whether higher-order interactions are *set-type* (arbitrary subsets, no hierarchy) or *hierarchical* (closed under faces/incidence). Hypergraphs represent set-type relations; simplicial and cell complexes represent hierarchical relations; combinatorial complexes aim to unify both [1, 5].

Table 1: Higher-order domains used in graph learning (high-level).

Domain	Core objects	Key property / modeling advantage
Graph	Nodes + Edges	Dyadic relations. Simplest propagation neighborhoods
Hypergraph	Nodes + Hyperedges	Polyadic relations without requiring lower-order faces
Simplicial complex	Simplices of all dimensions	Closure under faces. Canonical boundary/co-boundary structure
Cellular complex	General cells of various dimensions	Supports polygonal/mesh-like domains beyond simplices
Combinatorial complex	Ranked cells + Neighborhood functions	Unifies set-type and hierarchical relations; flexible multi-neighborhood design

3.2 Hypergraphs

Definition 3.1 (Hypergraph)**.** *A (finite) hypergraph is $H = (V, \mathcal{E})$ where V is a finite vertex set and $\mathcal{E} \subseteq \mathcal{P}(V) \setminus \{\emptyset\}$ is a set of hyperedges (subsets of arbitrary cardinality).*

Definition 3.2 (Incidence matrix)**.** *Let $n = |V|$ and $m = |\mathcal{E}|$. The incidence matrix $B \in \{0,1\}^{n \times m}$ is defined by $B(v, e) = 1$ if, and only if, $v \in e$. Weighted versions incorporate node and hyperedge weights, leading to generalized degree matrices and Laplacians that drive hypergraph diffusions [2].*

Definition 3.3 (Clique expansion)**.** *Given a hypergraph $H = (V, E)$, its clique expansion is the graph obtained by replacing each hyperedge $e \in E$ by the complete graph on the vertex set e, i.e., by adding all pairwise edges among vertices in e.*

Clique expansion is lossy, as it replaces each hyperedge by all pairwise edges among its nodes, collapsing distinct polyadic patterns. Many hypergraph message passing methods therefore operate directly on incidence or bipartite representations [1, 5].

Definition 3.4 (Bipartite graph)**.** *A graph $G = (U \sqcup W, F)$ is bipartite if its vertex set can be partitioned into two disjoint classes U and W such that every edge has one endpoint in U and the other in W.*

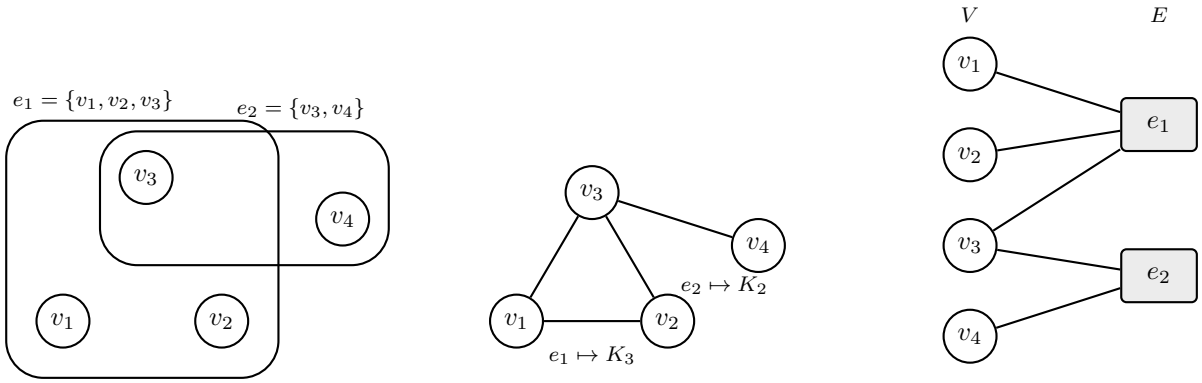
Definition 3.5 (Bipartite expansion)**.** *Given a hypergraph $H = (V, E)$, its bipartite expansion is the bipartite graph with vertex classes V and E, where $v \in V$ and $e \in E$ are adjacent if, and only if, $v \in e$. Equivalently, this incidence relation is encoded by the incidence matrix B.*

Example 3.1. *These definitions (3.1-3.5) can be better visualized in Figure 2.*

3.3 Simplicial complexes

Definition 3.6 (Simplicial complex [11])**.** *Let V be a non-empty set. A simplicial complex K is a collection of non-empty subsets of V such that:*

- *All singletons $\{v\}$ belong to K.*
- *If $\sigma \in K$ and $\emptyset \neq \tau \subseteq \sigma$, then $\tau \in K$.*



(a) Hypergraph (b) Clique expansion (c) Bipartite expansion

Figure 2: A hypergraph and two standard graphifications. Left: the original hypergraph with hyperedges $e_1 = \{v_1, v_2, v_3\}$ and $e_2 = \{v_3, v_4\}$. Middle: the clique expansion, where each hyperedge is replaced by a complete graph on its incident vertices. Right: the bipartite expansion, where vertices and hyperedges become two node classes connected by incidence edges.

A set σ with $|\sigma| = d + 1$ is a d-simplex. Thus, a 0-simplex is a vertex, a 1-simplex an edge, a 2-simplex a filled triangle, and a 3-simplex a filled tetrahedron. If $\tau \subset \sigma$, then τ is called a face of σ, and σ is called a coface of τ.

Simplicial complexes encode higher-order interactions *and* all lower-dimensional faces. This enables canonical boundary operators and rank-wise Laplacians (Hodge Laplacians) that are central in topological signal processing and simplicial message passing [5].

3.4 Cellular (CW) complexes (high-level)

Definition 3.7 (Cellular complex)**.** *A cellular (CW) complex is a space built from cells of different dimensions (vertices, edges, polygons, and higher-dimensional cells) that are attached to lower-dimensional cells via attaching maps. Unlike simplicial complexes, the cells need not be simplices, which makes cellular complexes suitable for polygonal or more general non-simplicial geometries.*

Cellular complexes generalize simplicial complexes and are useful for modeling domains with polygonal or more general non-simplicial geometry. In learning, cellular message passing often mirrors simplicial message passing, but uses cell-specific incidence structures.

3.5 Combinatorial complexes and neighborhood functions

A key idea in topological deep learning is to specify local interactions through *neighborhood functions* rather than a single adjacency. This supports multiple notions of locality (incidence-like, adjacency-like, path-based, etc.) within the same domain [5].

Definition 3.8 (Combinatorial complex [5, 6])**.** *A combinatorial complex is a triple (S, X, rk) where S is a finite vertex set, $X \subseteq \mathcal{P}(S) \setminus \{\emptyset\}$ is a set of cells, and $\mathrm{rk} : X \to \mathbb{Z}_{\geq 0}$ assigns a rank to each cell (rank-0 cells are singletons). Neighborhood functions define relations between cells, giving rise to one or more propagation operators.*

3.6 Relational structures as an alternative universal language

A different unifying language is *relational structures*: a set of entities together with relations of varying arity. This is the viewpoint used by Taha et al. [11] to connect simplicial/cellular message passing to graph-theoretic tools via influence graphs. It offers a "domain-agnostic" approach to modeling message passing, at the cost of choosing how to encode relations and shifts.

4 Higher-order message passing (HOMP): principles and templates

4.1 Carrier objects and rank-wise states

Higher-order message passing departs from node-only state representations. Depending on the domain, states can live on:

- Nodes only (graph-style).
- Nodes and hyperedges (incidence-based hypergraph networks).
- Simplices/cells at multiple ranks (complex-based networks).
- Ranked cells in a combinatorial complex (CCNN[1]/CTNN[2] style).

Thus, the main modeling choice is not only how to propagate information, but also which objects are allowed to carry hidden states.

4.2 General template

Let $\mathcal{X}$ denote the carrier set, i.e., the objects that carry embeddings. A generic higher-order message passing layer takes the form

$$m_x^{(\ell+1)} = \mathrm{AGG}_{y\in\mathcal{N}(x)}\, M_\ell\Big(h_x^{(\ell)},\, h_y^{(\ell)},\, \mathrm{rel}(x,y)\Big)\,, \tag{7}$$

$$h_x^{(\ell+1)} = U_\ell\Big(h_x^{(\ell)},\, m_x^{(\ell+1)}\Big)\,, \tag{8}$$

where $\mathcal{N}(x)$ is a neighborhood of x in $\mathcal{X}$, and $\mathrm{rel}(x,y)$ encodes the relevant higher-order relation between x and y (e.g., incidence type, shared faces/cofaces, or hyperedge metadata). This template subsumes many higher-order architectures once $\mathcal{X}$ and $\mathcal{N}$ are specified appropriately [1, 5].

4.3 Operator view: propagation + nonlinearity

As on graphs, many HOMP layers admit the schematic operator form

$$H^{(\ell+1)} = \sigma\Big(P\, H^{(\ell)} W_\ell\Big)\,, \tag{9}$$

[1] Combinatorial Complex Neural Networks.
[2] Copresheaf Topological Neural Networks.

where P is a propagation operator built from higher-order incidence, adjacency, or neighborhood relations. Depending on the architecture, P may be rank-specific or may combine several neighborhood channels. This is the point at which random walks and diffusion operators enter the picture.

4.4 Hasse graphs and multi-neighborhood decomposition

Simplicial, cellular, and combinatorial complexes can be associated with Hasse graphs (or incidence graphs), whose nodes are the cells of the complex. Two cells are connected whenever one is incident to the other as an immediate face or coface; equivalently, in a ranked setting, there is an edge between cells τ and σ when τ is contained in σ and their ranks differ by one. Thus, the Hasse graph makes explicit the local incidence structure across consecutive ranks.

This viewpoint is especially useful in learning, because it turns higher-order interactions into graph-like propagation channels. A recurrent strategy is to define one graph per neighborhood type (often an augmented Hasse graph), run a backbone model over each of them, and then aggregate the resulting representations rank-wise. This multi-neighborhood decomposition is central both conceptually, because it separates distinct interaction channels, and practically, because it allows one to reuse standard GNN backbones on higher-order domains.

5 Hypergraph message passing frameworks

5.1 Auxiliary graph expansions

Clique expansion and bipartite expansion, introduced in Section 3.2, are widely used to reduce hypergraphs to graphs. They are attractive computationally (reuse any GNN), but the reduction can lose polyadic information. Many hypergraph learning methods can be organized according to whether they operate directly on the hypergraph structure (hypergraph-native methods) or first transform the hypergraph into an auxiliary graph representation (graphified methods). [1, 5].

5.2 Incidence-based node-hyperedge-node message passing

A standard hypergraph-native paradigm alternates aggregation between nodes and hyperedges:

$$h_e^{(\ell+1)} = \phi_e\Big(h_e^{(\ell)},\ \mathrm{AGG}_{v\in e}\,\psi_{v\to e}(h_v^{(\ell)})\Big)\,, \tag{10}$$

$$h_v^{(\ell+1)} = \phi_v\Big(h_v^{(\ell)},\ \mathrm{AGG}_{e\ni v}\,\psi_{e\to v}(h_e^{(\ell+1)})\Big)\,, \tag{11}$$

with permutation-invariant aggregation inside hyperedges. This mirrors node-hyperedge-node random walks used to define hypergraph diffusion operators [2].

The node–hyperedge–node scheme can be interpreted as a local realization of a hypergraph propagation operator; diffusion-inspired layers make this operator explicit.

5.3 Diffusion-inspired hypergraph layers

Carletti et al. [2] derives a generalized random walk Laplace operator driven by hyperedge-mediated exchanges, reducing to the classical random-walk Laplacian in the graph case. Such operators

provide principled propagation matrices P for hypergraph message passing and emphasize that *hypergraph diffusion is not unique*.

Beyond the choice of a particular hypergraph Laplacian, diffusion-inspired hypergraph layers involve several modeling decisions: how to sample incident hyperedges (uniformly or with size/weight bias), how to choose destinations within a selected hyperedge, and whether to enrich the walk with non-backtracking constraints, restart, or multi-step kernels. These choices modify the effective propagation operator P and the geometry it induces on the hypergraph, thereby affecting which structural patterns or roles can be captured [3].

6 Simplicial and cellular message passing frameworks

Simplicial and cellular complexes differ from ordinary graphs and hypergraphs in that they support several distinct local relations between cells, both across ranks and within the same rank. As a result, message passing is no longer governed by a single neighborhood notion, but by multiple propagation channels induced by incidence, co-incidence, and adjacency structure. In this section, we first summarize the basic local relations on simplicial complexes, then explain how they lead to cross-rank transport and rank-wise Hodge-type operators, and finally discuss the alternative relational viewpoint of influence graphs.

6.1 Boundary, co-boundary, and adjacency relations on simplicial complexes

Following Taha et al. [11], for a simplex σ one considers several local relations:

- Boundary relation $B(\sigma)$: the set of faces of σ.
- Co-boundary relation $C(\sigma)$: the set of cofaces of σ.
- Lower adjacency $N_{\downarrow}(\sigma)$: simplices of the same rank that share a common face.
- Upper adjacency $N_{\uparrow}(\sigma)$: simplices of the same rank that share a common coface.

The first two relations connect simplices across ranks, whereas the latter two define same-rank neighborhoods. Simplicial message passing aggregates through one or several of these channels and updates simplex features rank-wise. These local relations can be encoded by incidence operators and their adjoints, which in turn induce rank-wise diffusion operators.

6.2 Cross-rank transport and rank-wise Hodge operators (high level)

Simplicial and cellular complexes support boundary operators mapping k-cells to $(k-1)$-cells, together with their adjoints, which transport information across ranks. From these incidence operators one builds rank-specific Laplacians (Hodge Laplacians), which govern within-rank diffusion of cochains and enable learning from edge-, face-, and higher-cell signals [5]. While we do not develop the full Hodge-theoretic formalism here, this viewpoint clarifies why complex-based message passing is intrinsically multi-rank.

6.3 Relational structures and influence graphs

Returning to the relational viewpoint introduced in Section 3.6, Taha et al. [11] interpret simplicial and cellular message passing as message passing over relational structures. A key construct is an influence graph, which records how information flows through the relational architecture and thereby makes it possible to import graph-theoretic analyses (including rewiring heuristics and oversquashing diagnostics) beyond ordinary graphs. This relationalization viewpoint will later be compared with intrinsic propagation defined directly from higher-order connectivity.

7 Combinatorial complexes, CCNNs, and TopoTune/GCCNs

7.1 CCNNs as a unifying higher-order message passing class

Building on the multi-rank viewpoint of the previous section, Hajij et al. [5] introduce combinatorial complex neural networks (CCNNs), a general framework for higher-order message passing built from three ingredients: combinatorial complexes, neighborhood functions defining local interactions, and tensor operations such as push-forward and merge. This viewpoint provides an abstract language in which many seemingly different higher-order architectures can be expressed within a common design framework. In this sense, CCNNs are not just a particular model class, but a unifying formalism for a broad family of higher-order message passing networks. This idea is sometimes summarized informally as "HOMPNNs are CCNNs" [5].

7.2 TopoTune: generalized CCNNs (GCCNs) as a modular framework

Papillon et al. [10] propose generalized CCNNs (GCCNs) and the TopoTune framework. Given a combinatorial complex and a collection of neighborhood functions, TopoTune expands the complex into multiple augmented Hasse graphs, one for each neighborhood, applies a base architecture (such as a GNN, Transformer, or MLP) to each neighborhood-specific expansion, and then aggregates the resulting features rank-wise. This construction turns the CCNN viewpoint into a systematic and modular design pipeline for topological deep learning architectures, bringing higher-order models closer to the plug-and-play ecosystem of graph learning [10].

8 Sheaves, copresheaves, and CTNNs

8.1 Sheaf and copresheaf transport as directional propagation

Unlike isotropic message passing, sheaf-like models attach vector spaces to cells and linear maps to relations, so that information is transported through learned directional maps rather than merely averaged. This yields anisotropic and relation-specific propagation, allowing the model to adapt how signals move across the underlying structure. In particular, such transport mechanisms can help mitigate heterophily and oversmoothing by going beyond uniform neighborhood aggregation [6]. More broadly, sheaf and copresheaf viewpoints provide a natural language for directional information flow on higher-order domains, where the interaction between two cells is mediated not only by connectivity but also by the map relating their local representation spaces.

8.2 CTNNs: copresheaf topological neural networks

Building on this perspective, Hajij et al. [6] introduce copresheaf topological neural networks (CTNNs), a unifying framework based on copresheaves over combinatorial complexes. The key idea is to allow each local region or cell to have its own representation space, together with learnable copresheaf maps $\rho_{x_i \to x_j}$ governing transport between related cells. This generalizes neighborhood-based higher-order message passing by replacing purely combinatorial aggregation with structured transport across local spaces. The framework is presented by the authors as encompassing GNNs, attention mechanisms, sheaf diffusion models, and other topological neural architectures within a single formalism [6].

9 Random walks and diffusion operators as propagation choices

Random walks and diffusion kernels are not only tools for network analysis; in modern graph learning they are natural choices for the propagation operator P in message passing layers. From this viewpoint, changing the walk amounts to changing the geometry through which information is mixed, and hence the trade-off between locality, long-range communication, and smoothing.

9.1 Random walks on graphs

On ordinary graphs, standard propagation operators already arise from random walks and diffusion kernels. Let G be a connected undirected graph with adjacency matrix A and degree matrix $D = \mathrm{diag}(d_1, \ldots, d_n)$. The simple random-walk transition matrix is

$$P = D^{-1}A,$$

whose stationary distribution is proportional to node degrees in the undirected case.

Common graph-level variants include:

- **Lazy random walk:**
$$P_{\text{lazy}} = \tfrac{1}{2}(I + P),$$
meaning that at each step the walker either stays at the current node or follows the usual random walk. This damps oscillations, slows mixing, and often yields a more stable diffusion.

- **Weighted random walk:**
$$P = D_W^{-1}W,$$
where W is a nonnegative edge-weight matrix and D_W is the corresponding diagonal strength matrix. Here transitions are biased toward neighbors connected by larger weights, so propagation reflects edge intensities or affinities rather than mere binary connectivity.

- **Random walk with restart / personalized PageRank:**
$$K_\alpha = (1-\alpha)\sum_{k\geq 0} \alpha^k P^k,$$
which can be interpreted as a walk that repeatedly diffuses but, with fixed probability, returns to its starting point. This balances multi-step propagation with locality, preventing the walk from drifting too far from its source.

- **Multi-step kernels:** powers P^k or mixtures

$$K = \sum_k \beta_k P^k,$$

  which explicitly incorporate paths of multiple lengths. In this way, one can control how much weight is given to short-range versus longer-range interactions, thereby tuning the effective receptive field of the propagation operator.

These operators control locality versus long-range mixing and are closely tied to oversmoothing phenomena in deep GNNs [13].

9.2 Random walks on hypergraphs

In hypergraphs, unlike in ordinary graphs, there is no unique canonical random walk. Different choices of how to traverse hyperedges lead to different propagation operators and therefore to different induced geometries.

A representative intrinsic construction is given by Carletti et al. [2], who define random walks from a microscopic exchange model: a walker first selects an incident hyperedge, with probabilities that may depend on hyperedge sizes or weights, and then jumps to a node within that hyperedge. This produces a generalized random-walk Laplacian that reduces to the classical graph random-walk Laplacian when all hyperedges have size 2.

This viewpoint is especially relevant for message passing: once the walk is fixed, the associated transition or diffusion operator provides a principled propagation matrix P for hypergraph layers. Moreover, the design choices in the walk (for example, whether to bias hyperedge sampling, whether to use restart, or whether to incorporate multi-step kernels) can substantially change the geometry seen by the model [2, 3].

9.3 Random walks on complexes and combinatorial complexes

On simplicial, cellular, and combinatorial-complex domains, random-walk and diffusion viewpoints can be instantiated on several supporting structures.

- **Hasse or incidence graphs:** one can walk on cells or incidences after graphification.
- **Cochain spaces:** one can define diffusion through rank-specific Laplacians together with boundary/co-boundary couplings, yielding multi-rank propagation.
- **Neighborhood-function-induced graphs:** in the combinatorial-complex viewpoint, one may define one propagation channel per neighborhood function and then combine them.

The last perspective gives an operator-level interpretation of the multi-neighborhood decomposition used in CCNNs and GCCNs [5, 10]. More generally, across higher-order domains, the walk is not merely an implementation detail: it is part of the model design. Thus, across higher-order domains, choosing a walk is equivalent to choosing an induced propagation geometry, and hence part of the model design rather than a downstream implementation detail.

10 Structural roles and equivalences: why the hypergraph walk matters

The previous section showed that higher-order propagation depends on the choice of walk or diffusion operator. This matters not only for smoothing and receptive fields, but also for *which nodes or cells are regarded as behaviorally similar*. In many applications, the relevant notion of similarity is not proximity, but *role*.

10.1 Roles beyond proximity

On ordinary graphs, *structural equivalence* requires two nodes to have identical neighborhoods, whereas *regular equivalence* is a coarser notion that identifies nodes occupying similar relational positions even if their neighbors are not literally the same. These notions are role-based rather than distance-based: they seek similarity of function or position in the network rather than mere closeness.

This distinction is important for graph learning because two nodes may be far apart in the graph yet play the same structural role, while nearby nodes may belong to very different roles. Accordingly, a propagation operator designed only for short-range averaging may fail to preserve the distinctions relevant for role-sensitive tasks. Thus, the relevant question is not only how far information propagates, but which structural roles remain stable under the chosen diffusion geometry

10.2 Hypergraph geometries and approximate role equivalence

Eidi and Otter [3] generalize structural and regular equivalence to undirected hypergraphs and study how neighborhood-graph constructions and random-walk-induced geometries relate to approximate role equivalences. Their viewpoint suggests that the geometry induced by a hypergraph walk is not neutral: it shapes which nodes appear similar and which distinctions remain visible after diffusion.

This gives a concrete motivation for designing hypergraph propagation operators beyond one-step incidence-based baselines. Variants involving restart, multi-step kernels, or interpolations between local choices may capture structural roles better than a single local walk. From the perspective of message passing, the hypergraph walk therefore acts as a genuine modeling choice, not just a technical detail.

In this sense, the question of *which* hypergraph random walk to use is directly connected to the broader guiding question of the survey: whether higher-order propagation should be defined intrinsically from hypergraph connectivity or via auxiliary relational constructions, and how these choices affect distinguishability and role structure [2, 3].

11 Synthesis and outlook

11.1 A unifying "design knob" list

Across the surveyed frameworks, the main design knobs are:

1. **Domain**: hypergraph vs simplicial/cellular vs combinatorial complex vs relational structure.

2. **Carrier ranks**: node-only vs multi-rank embeddings.

3. **Neighborhoods**: incidence/boundary/co-boundary, lower/upper adjacency, CC neighborhoods, walk-based neighborhoods.

4. **Propagation operator**: one-step vs multi-step; restart vs no restart; size-weighted vs unweighted; isotropic vs directional transport.

5. **Aggregation/transport**: invariant pooling vs attention vs sheaf/cosheaf maps.

11.2 Returning to the open question

The survey highlights the conceptual tension motivating the subsequent project:

- **Intrinsic hypergraph propagation:** define message passing directly from hyperedge connectivity patterns and hypergraph-specific random walks [2, 3].
- **Relationalization:** define propagation through relational structures, influence graphs, or auxiliary graph representations, as in the framework of Taha et al. [11].

A first next step is to review standard random walk variants on graphs (including lazy walks, weighted walks, restart/personalized PageRank, and multi-step kernels) and to formulate and compare their natural lifts to higher-order domains. In particular, the project will study how such variants can be transferred to hypergraphs through hyperedge-sensitive walk constructions, and how analogous diffusion viewpoints appear on simplicial, cellular, and combinatorial-complex domains.

The broader goal is then to investigate whether one can define a genuinely hypergraph-native message passing paradigm based on hyperedge connectivity patterns and random-walk-derived propagation operators, rather than relying on relational graph constructions. These intrinsic operators will be compared with relationalized alternatives in terms of the geometries they induce, the structural roles they preserve, and their effect on distinguishability and related diffusion-based measures.

Acknowledgements

This draft is prepared as part of a research project supervised by Prof. Guido Montúfar.

References

[1] Maciej Besta, Florian Scheidl, Lukas Gianinazzi, Grzegorz Kwasniewski, Shachar Klaiman, Jürgen Müller, and Torsten Hoefler. Demystifying higher-order graph neural networks. *arXiv preprint*, 2025. arXiv:2406.12841v4.

[2] Timoteo Carletti, Federico Battiston, Giulia Cencetti, and Duccio Fanelli. Random walks on hypergraphs. *arXiv preprint*, 2019. arXiv:1911.06523v2.

[3] Marzieh Eidi and Nina Otter. Geometric characterisation of structural and regular equivalences in undirected (hyper)graphs. *arXiv preprint*, 2025. arXiv:2512.24961v1.

[4] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1704.01212.

[5] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Nina Miolane, Aldo Guzmán-Sáenz, Karthikeyan Natesan Ramamurthy, Tolga Birdal, Tamal K. Dey, Soham Mukherjee, Shreyas N. Samaga, Neal Livesay, Robin Walters, Paul Rosen, and Michael T. Schaub. Topological deep learning: Going beyond graph data. *arXiv preprint*, 2023. arXiv:2206.00606v3.

[6] Mustafa Hajij, Lennart Bastian, Sarah Osentoski, Hardik Kabaria, John L. Davenport, Sheik Dawood, Balaji Cherukuri, Joseph G. Kocheemoolayil, Nastaran Shahmansouri, Adrian Lew, Theodore Papamarkou, and Tolga Birdal. Copresheaf topological neural networks: A generalized deep learning framework. *arXiv preprint*, 2025. arXiv:2505.21251v3.

[7] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pages 1025–1035. Curran Associates, Inc., 2017. ISBN 9781510860964.

[8] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.02907.

[9] Haggai Maron, Heli Ben-Hamu, Nadav Shamir, and Yaron Lipman. Invariant and equivariant graph networks. In *International Conference on Learning Representations (ICLR)*, 2019. Published as a conference paper at ICLR 2019.

[10] Mathilde Papillon, Guillermo Bernardez, Claudio Battiloro, and Nina Miolane. Topotune: A framework for generalized combinatorial complex neural networks. *arXiv preprint*, 2025. arXiv:2410.06530v4.

[11] Diaaeldin Taha, James Chapman, Marzieh Eidi, Karel Devriendt, and Guido Montúfar. Demystifying topological message-passing with relational structures: A case study on oversquashing in simplicial message-passing. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2506.06582v1.

[12] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.10903.

[13] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386. arXiv:1901.00596.

[14] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.00826.

A Derivation of the symmetric normalized Laplacian

A standard normalization (especially useful in spectral derivations) is obtained by scaling L_{un} on the left and right by $D^{-1/2}$:

$$L_{\text{sym}} := D^{-1/2} L_{\text{un}} D^{-1/2} = D^{-1/2}(D - A)D^{-1/2} = I - D^{-1/2} A D^{-1/2}.$$

This L_{sym} is the (symmetric) normalized Laplacian, often denoted L_{norm}.

Intuition (degree-normalization): The factor $D^{-1/2}$ degree-normalizes node-wise signals: it rescales a vector $x \in \mathbb{R}^n$ to $\tilde{x}$ with $\tilde{x}_i = x_i/\sqrt{d_i}$. This compensates for degree heterogeneity, so high-degree nodes do not dominate simply because they have more neighbors.